

CSA1717-ARTIFICIAL INTELLIGENCE
PRACTICAL LAB EXPERIMENTS
RAGAVI S – (192565027)

EXPERIMENT 10: A* ALGORITHM

Aim

To write a python program to implement A* Algorithm.

Objective

- To understand the concept of the A* (A-Star) search algorithm.
- To implement the A* algorithm in Python.
- To find the shortest path from a start node to a goal node using heuristic values.

Algorithm

Step1 :Initialize the open list with the starting node and the closed list as an empty set.

Step 2: Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

Step 3: While the open list is not empty, select the node with the lowest f score and set it as the current node.

Step 4: If the current node is the goal node, return the reconstructed path. Remove the current node from the open list and add it to the closed list.

Step 5: For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

Step 6: If the neighbor is already in the closed list, skip it.

Step 7: If the neighbor is not in the open list, add it and set its g score and f score.

Step 8 :If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

Step 9 : Otherwise, update the came from dictionary, g score, and f score of the neighbor.

Step 10 : Repeat steps 3-10 until the open list is empty.

If the goal node was never reached, return failure.

Source Code

```
import heapq

def a_star(graph, heuristic, start, goal):
    # Priority queue: (f, g, node, path)
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))

    visited = set()

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        if current == goal:
            return path, g

        if current in visited:
            continue

        visited.add(current)

        for neighbor, cost in graph[current].items():
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    open_list,
```

```

        (new_f, new_g, neighbor, path + [neighbor])
    )

return None, float('inf')

# Input
n = int(input("Enter number of nodes: "))

graph = {}

print("Enter graph as: number_of_neighbors neighbor cost ...")

for i in range(n):
    parts = list(map(int, input(f"Node {i}: ").split()))
    num_neighbors = parts[0]
    graph[i] = {}

    index = 1
    for _ in range(num_neighbors):
        neighbor = parts[index]
        cost = parts[index + 1]
        graph[i][neighbor] = cost
        index += 2

print("Enter heuristic values:")
h_values = list(map(int, input().split()))

heuristic = {i: h_values[i] for i in range(n)}

```

```
start = int(input("Enter start node: "))
```

```
goal = int(input("Enter goal node: "))
```

```
path, total_cost = a_star(graph, heuristic, start, goal)
```

```
if path:
```

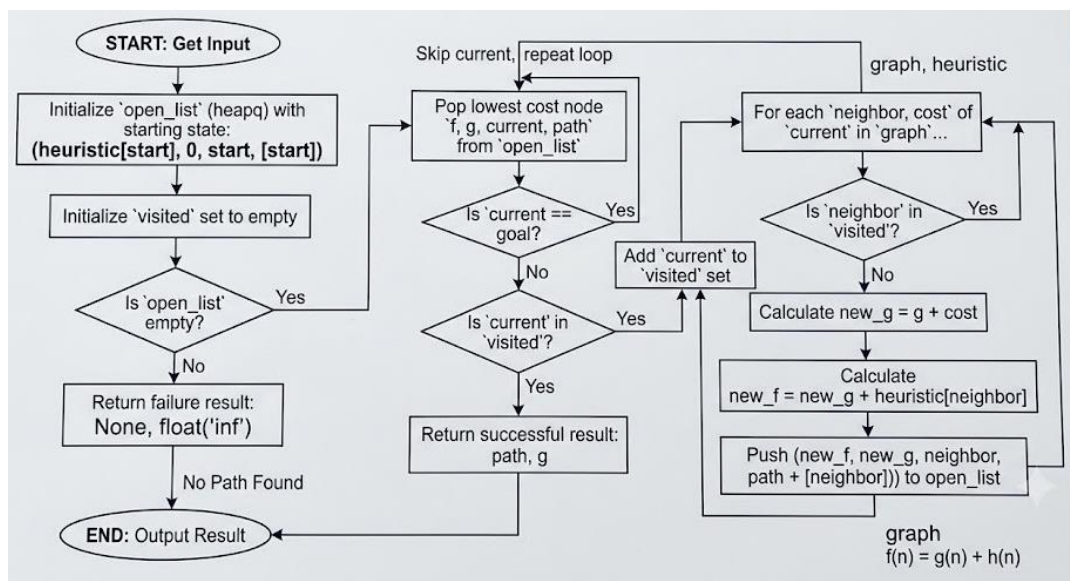
```
    print("\nPath found:", " -> ".join(map(str, path)))
```

```
    print("Total cost:", total_cost)
```

```
else:
```

```
    print("\nNo path found!")
```

Flowchart



Output

```
>>>| ===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/Q.py =====  
| Enter number of nodes: 4  
| Enter graph as: number_of_neighbors neighbor cost ...  
| Node 0: 2 1 1 2 4  
| Node 1: 1 3 2  
| Node 2: 1 3 1  
| Node 3: 0  
| Enter heuristic values:  
| 7 6 2 0  
| Enter start node: 0  
| Enter goal node: 3  
|  
| Path found: 0 -> 2 -> 3  
| Total cost: 5  
>>>|
```

Result

The A* search algorithm was successfully implemented in Python. The program calculated the optimal path from the start node to the goal node using the given heuristic values and displayed the total traversal cost.